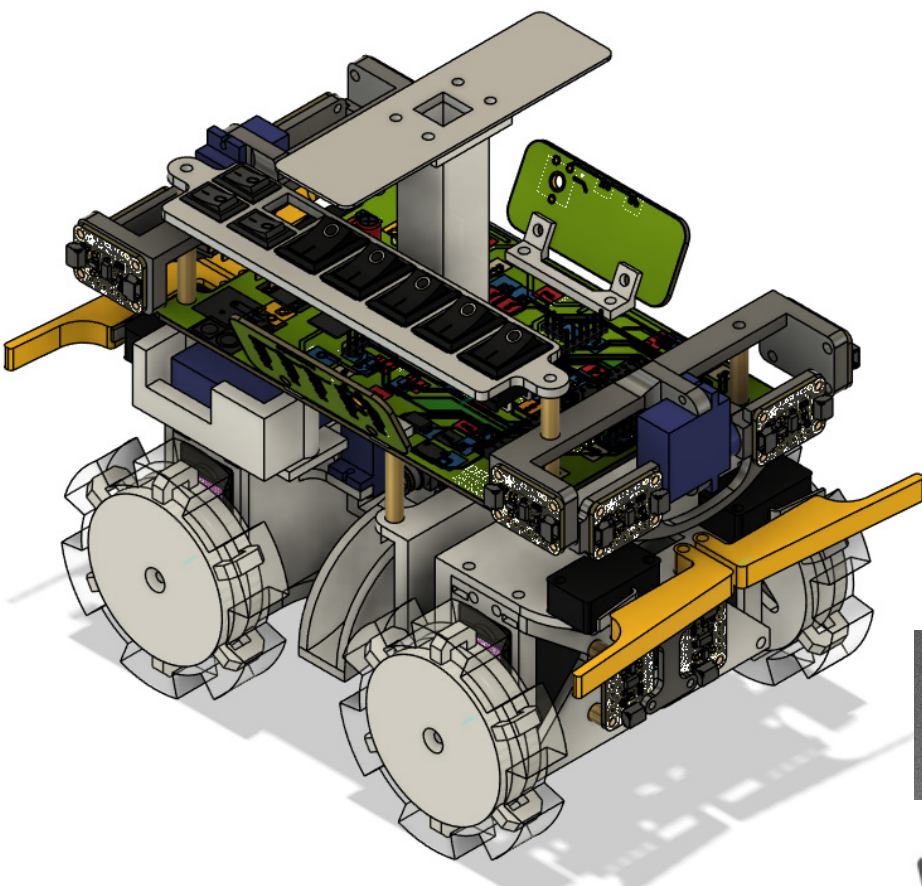


Method, Production & Design

Robot Construction



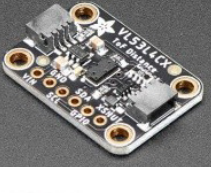
The structural design and PCBs of the robot were created in Autodesk Fusion. Using Autodesk Fusion allowed for the robot to be visualised at any stage in the design before manufacturing it. The robot was designed and manufactured completely in-house with the exception of the PCBs, which were designed in-house but sent to JLC PCB to manufacture. The PCBs were still hand-soldered in-house. This robot used many manufacturing techniques, including 3D printing on a Bambu Lab P1S and CNC milling on a Makera Carvera. The current robot uses nine major components, these can be seen on the right.



Teensy 4.1:
The main controller that does all the processing for the robot is a teensy 4.1. This was chosen due to its superior speeds compared to other microcontrollers and its small form factor.



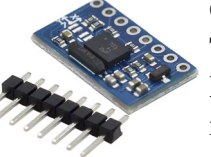
MG996R Continuous Servo Motor:
The robot holds four MG996R continuous servo motors to drive the wheels. These were chosen due to their ease of use and relatively small form factor.



Adafruit VL53L4CX:
The robot holds two VL53L4CX distance sensors for long distance sensing in large open spaces of mazes.



Limit Switches:
The robot uses four limit switches, two on the front and two on the rear. These are used for centering the robot and detecting obstacles.



GY-BNO055 9 axis IMU:
The robot uses one BNO055 IMU for detecting the rotation of the robot in three axis.



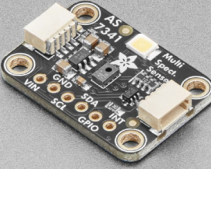
OpenMV AE3:
The robot holds two OpenMV AE3 cameras, one on the left and one on the right. These cameras are used for victim detection and were chosen for the small form factor.



Adafruit VL53L4CD:
The robot holds eight VL53L4CD distance sensors, two on each side. These are used for sensing walls in all directions and correcting the robots' skew while driving around the maze.



180-degree 9g Servo Motor:
The robot uses three 9g servo motors, two for the LRF pivot and one for the package delivery system.



Adafruit AS7341 colour sensor:
The robot holds one colour sensor on the bottom of the front for detecting colour floors such as red, black or blue. This sensor was chosen based on its large range of colour channels (10 channels).



Abstract

DELTA is an Australian team from Brisbane Boys' College competing in Robocup Junior Rescue Maze. We are a team of three members and have competed in the Rescue Maze category for two years. Development of the current robot started in late 2025 where every team member has put in over 300 hours each to design, build, program and test our robot. DELTA competed in the Robocup Junior Rescue Maze International competition in Salvador, Brazil in 2025. We have since competed in and achieved first place in the Robocup Junior Open Rescue Maze National Competition in 2025 and the Robocup Junior Open Rescue Maze Brisbane Regional Competition in 2026. The robot was made using industry standard technologies including - 3D printing, CNC Milling and laser cutting, along with the use of printed circuit boards (PCBs) to improve reliability and save time when constructing spare boards. Our team is made up of three members, Ethan Seymour, James Cousins, and Connor Duncan. Ethan works on the hardware, including electrical design, structural design and manufacturing. Ethan has designed every part of the current robot himself and has manufactured most parts himself with a CNC machine or 3D printer. James works on the main software and navigation algorithm and has coded every part of the teensy code himself. Connor works on the victim detection software and has trained and coded the cognitive target recognition himself.



Image of team DELTA at the Robocup Junior National competition in 2025

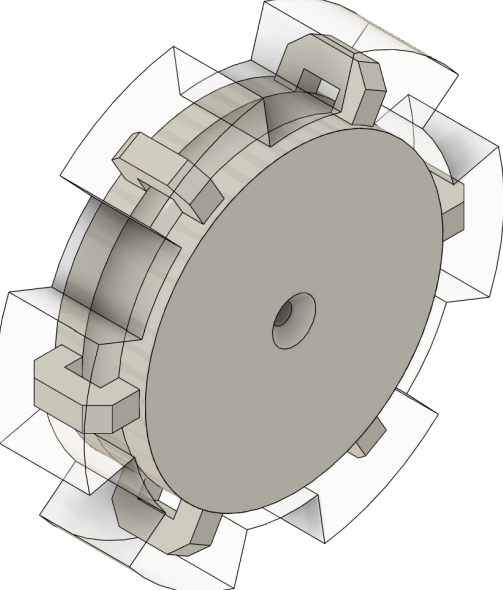


Image of team DELTA at the Robocup International competition in 2025



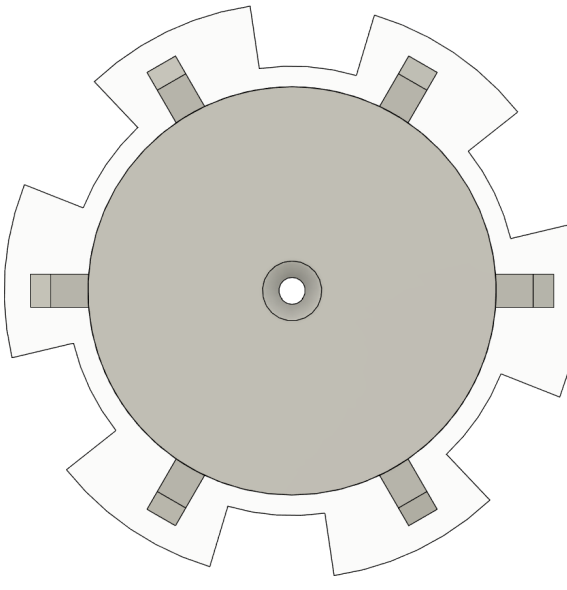
Scan the QR code to visit our website for more information.

Custom Silicone Wheels



The robot runs four of the same custom molded silicone wheels. These wheels are made from a central 3D printed hub made from PLA and Shore 25A silicone for the tire. Silicone was chosen because of its superior grip due to its elasticity. An added benefit of the silicone wheels is that unlike some other choices, silicone does not stick to most things and therefore will not pick up debris while navigating around the maze. The chosen silicone has a hardness of 25A on the Shore Hardness Scale and was chosen for its durability and grip. Other options were considered but were found to be too easily broken or not grippy enough. The tire incorporates six large notches that allow for the robot to climb stairs. The grooves of the stairs catches on the notches and gives the robot additional grip while climbing stairs.

Images of these wheels can be seen on the left and right.



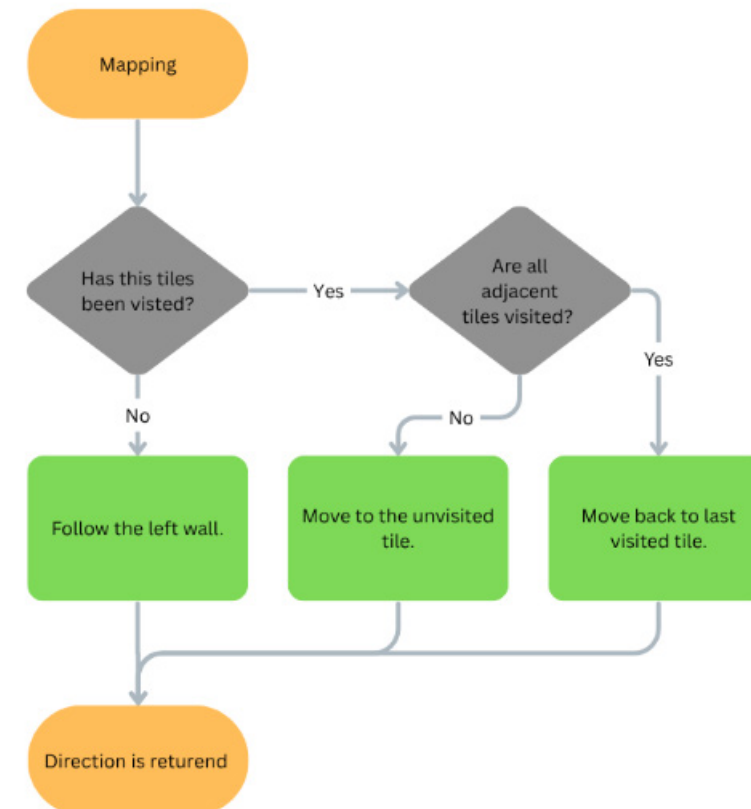
Software

Navigation Algorithm



The core architecture of the software relies on a Finite State Machine (FSM) to manage all robot movements. An FSM is a system where each task a robot needs to complete is given a state. When the robot then needs to complete that task, it will then move into that state. To then move out of this state there is then a set of conditions that need to be met for it to change into a different state. At the center of our FSM is the mapping state; as the robot traverses each tile, it executes the mapping algorithm to determine its next destination based on available empty tiles. This FSM is also in charge of the process for when a victim is detected and along with what task the robot is to perform on a tile such as a blue tile.

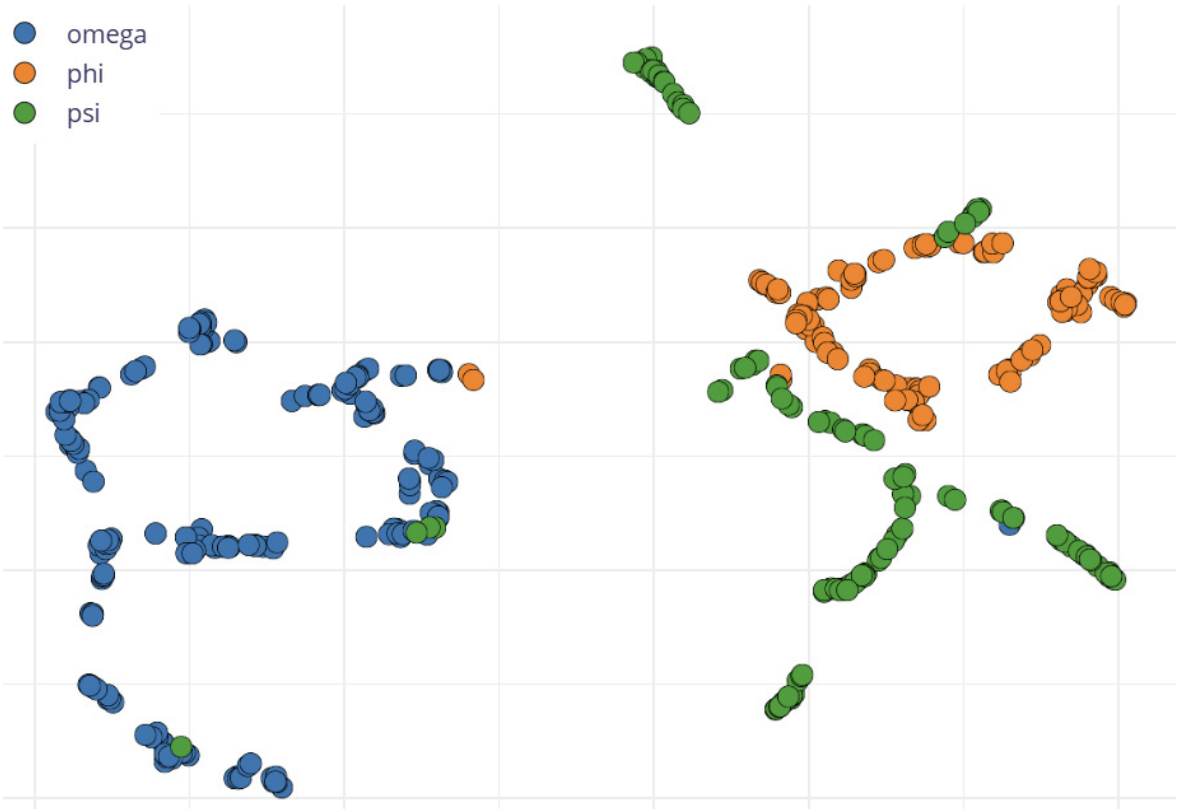
To ensure precise timing, sensor data is captured using hardware interrupts. The main software cycle runs every 100 milliseconds, during which different sensors are sampled at varying frequencies depending on their individual reading acquisition times. With different sensors being read every 5ms with the over 100ms sensor reading cycle.



Our mapping strategy combines a standard left-wall-following approach with a custom navigation algorithm. The robot follows the left wall while traversing unvisited tiles. However, upon reaching a previously visited tile, the custom algorithm takes over to determine the next move. This algorithm checks if any adjacent tiles connected to the current visited tile remain unvisited. If an unvisited neighbour is found, the robot navigates to it and resumes the left-wall-following algorithm. If all adjacent tiles have already been visited, the robot backtracks to the previous tile and repeats the check. This backtracking loop continues until an unexplored path is found and eventually the robot returns to the start tile. Throughout this process, crucial data such as tile coordinates, coloured tiles (e.g. blue or black tiles), and connectivity to other tiles.

To optimize memory usage, the robot implements dynamic memory allocation for mapping instead of using a 3D array or static list. This approach ensures the robot only consumes the exact amount of memory required, preventing waste. Furthermore, utilizing dynamic allocation shifts the data storage from the stack to the heap. This is crucial for the Teensy 4.1, which limits stack size to 16-32 KB while providing a much larger ~512 KB heap. Within this heap space, each map tile is represented as a structured data type (struct). Each tile struct stores its unique ID, coordinates, colour data (e.g., blue, black, silver), and victim presence, alongside pointers to adjacent tile structs to form the map network.

Victim Identification



To detect the letter victims we are using a FOMO MobileNetV2 object detection neural-network for both the letter victims and the cognitive targets. This approach was chosen after seeing its capabilities in 2025 at internationals where it was observed that many teams that used a similar approach achieved very good results and stood out for their victim identification. This year over 500 images were taken on an OpenMV AE3 camera of all the possible letter victims and these were used to train a model inside of Edge Impulse. Using the OpenMV AE3 camera helped replicate what the victims would look like on the robot by using the exact same hardware as mounted on the robot (same camera and lens). These images were then trained in Edge Impulse with 50 epochs, a learning rate of 0.003 and a validation set size of 20%. This was tuned from the default values to achieve a higher precision score, recall score and therefore a higher F1 score.

The image on the left shows the generated image features.

The target detection works in two steps, it first uses blob detection to locate a target by looking for a large area of colour. Then it uses ring detection code, which works in two main stages, to identify and interpret the target. The camera searches for the target area, locks onto the outer circle, and keeps that circle position stable across frames so the readings do not jump around from one frame to the next. After the target has been located, the code reads the five concentric rings by sampling several fixed points around each ring in the form of dots and checking which colour threshold each point matches. It then uses a voting method so that each ring gets a final colour decision based on the majority of samples. Once all five rings have been classified, the script converts those colours into their assigned numbered values, adds them together, and uses the total to decide whether the target is real and if it should drop a package or not. The code also checks that the same ring pattern is seen repeatedly before confirming the result, which helps reduce false detections.

An image of this software detecting a target can be seen below

